

Departmental Budget Management System

Project Report

Princess Nhyira Addai & J'Isabelle Umuvandimwe

Repository: github.com/Pincessis17/ProgrammingII_Summative

1. Overview

The Departmental Budget Management System is a JavaFX desktop application connected to a MySQL database using JDBC. It allows university administrators to manage departments, employees, and departmental budgets. The system follows a layered architecture consisting of the JavaFX UI, model, and DAO layers. Departments can contain employees and sub-departments, forming a hierarchical structure that supports the recursive algorithms and Composite Pattern used in the system.

2. Design Patterns

2.1 Composite Pattern

`OrgUnit` is the common interface. It defines:

- `getName()`
- `calculateBudget()`
- `getHeadcount()`

`Employee` is a leaf because it cannot contain other `OrgUnit` objects.

`Department` is a composite because it can contain:

- Employees
- Other Departments

Recursion allows `calculateBudget()` and `getHeadcount()` to work across the entire organisational hierarchy, regardless of whether the structure contains individual employees or nested departments.

This pattern is suitable because the organisation has a tree-like structure, where departments can contain sub-departments and employees.

2.2 DAO (Data Access Object)

DAO interfaces (`DepartmentDAO`, `EmployeeDAO`, `BudgetDAO`) define the operations required for each entity, such as `create()`, `findAll()`, `update()`, and `delete()/deactivate()`.

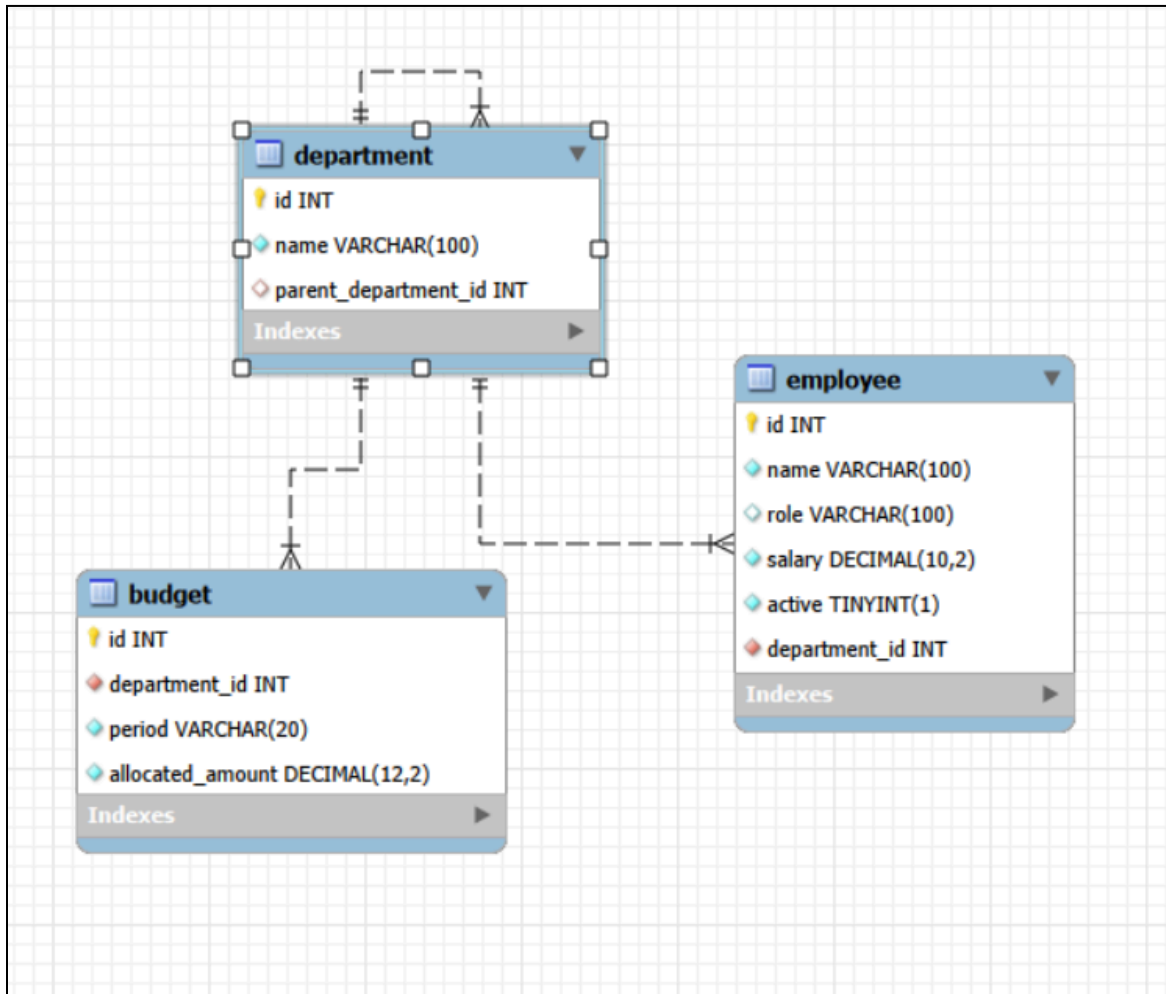
DAO implementations (`DepartmentDAOImpl`, `EmployeeDAOImpl`, `BudgetDAOImpl`) handle the actual SQL and database operations.

Controllers depend only on the DAO interfaces, not directly on the JDBC implementations.

This separates UI logic from database logic, allowing the database or storage method to be changed without modifying the controllers or FXML files.

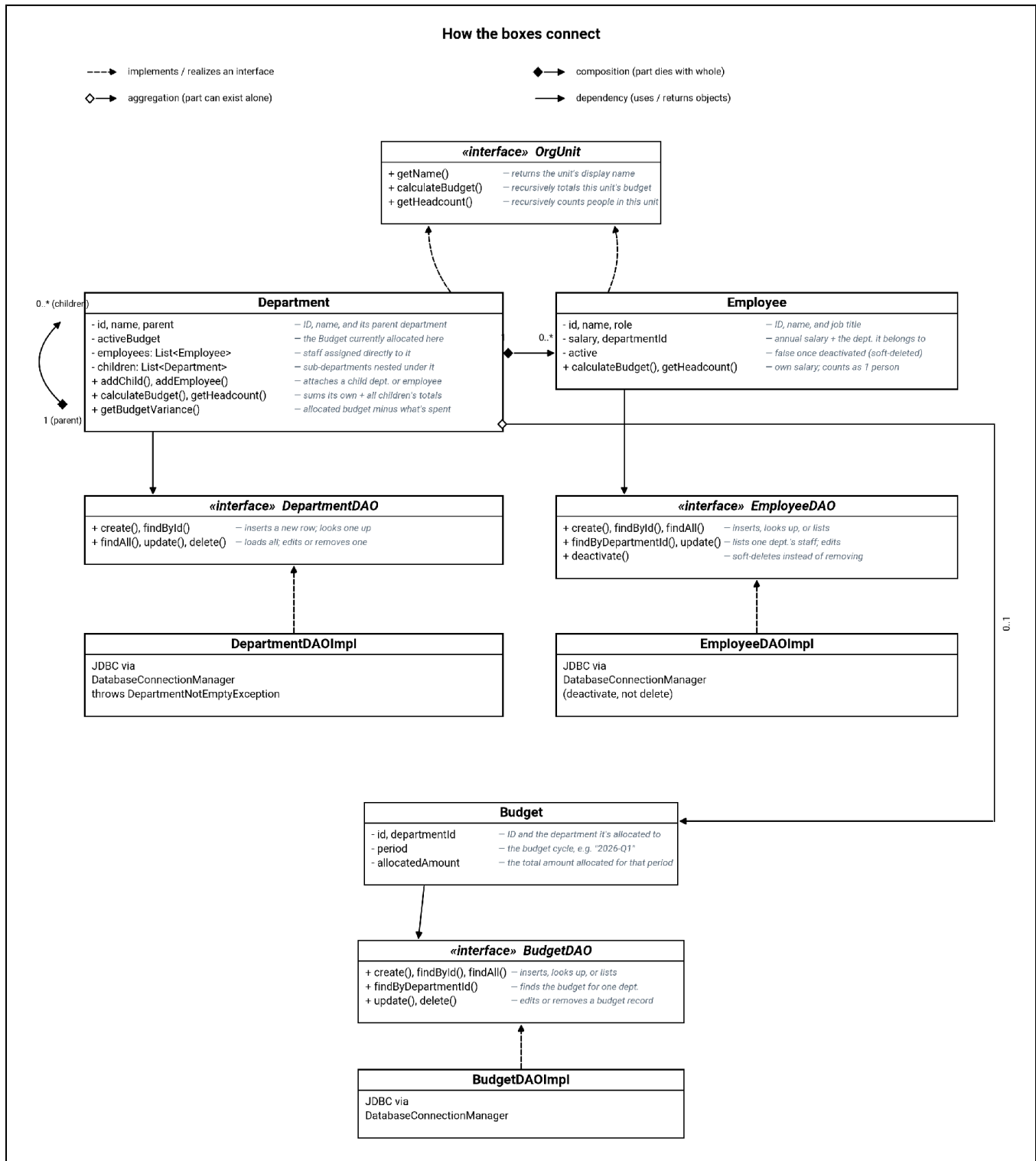
3. Data Model

3.1 Entity-Relationship Diagram



Department is self-referencing through `parent_department_id` (one parent, many children), and is the parent side of one-to-many relationships with both `employee` and `budget`.

3.2 Class Diagram



4. Recursive Algorithm

Where it lives: `Department.calculateBudget()` and `Department.getHeadcount()`, in `com.budgetms.model.Department`.

Because a department can contain employees and sub-departments, calculating its total budget or headcount requires traversing the entire department hierarchy. Both methods first calculate the department's direct contribution and then recursively process each child department.

```
public class Department implements OrgUnit { 2 Princess Nhyira Addai +1

    @Override 3 usages 2 Princess Nhyira Addai +1
    public double calculateBudget() {
        double total = 0;

        for (Employee employee : employees) {
            total += employee.calculateBudget();
        }

        for (Department child : children) {
            total += child.calculateBudget();
        }

        return total;
    }

    @Override 2 usages 2 Princess Nhyira Addai
    public int getHeadcount() {
        int total = 0;

        for (Employee employee : employees) {
            total += employee.getHeadcount();
        }

        for (Department child : children) {
            total += child.getHeadcount();
        }

        return total;
    }
}
```

Base case: A department with no child departments stops the recursion and returns the result based on its own employees and budget.

Recursive case: For each child department, the method calls itself and adds the child's result to the total.

Recursion limit: `Department.addChild()` limits the hierarchy to `MAX_DEPTH = 6`, preventing excessively deep recursion and ensuring the calculations terminate safely.

4.1 Recursive vs. Iterative

Recursive approach: Each department calculates its own total and recursively calculates the totals of its child departments.

Iterative approach: The same traversal could be performed using an explicit stack or queue to visit each department.

Why recursion was chosen:

- It directly matches the hierarchical structure of departments and sub-departments.
- `MAX_DEPTH = 6` limits recursion depth, so stack overflow is not a practical concern.
- It requires less additional bookkeeping than an explicit stack or queue.
- It keeps the traversal logic within `Department`, supporting the Composite Pattern by allowing the client to call one method on the root department to calculate the whole tree.

5. Testing

The project has 34 automated tests across 6 JUnit 5 test classes, run through the maven-surefire-plugin (`mvn test`). All 34 currently pass.

Test class	Tests	What it covers
<code>ValidationTest</code>	8	<code>ValidationUtils.requireNonBlank()</code> / <code>requirePositiveNumber()</code> ; <code>Department.addChild()</code> <code>MAX_DEPTH</code> boundary (7-level chain rejected, normal nesting accepted)
<code>DepartmentDaoTest</code>	7	<code>DepartmentDAOImpl</code> CRUD against the database: create, <code>findById</code> (existing + missing), update, delete, and delete-blocked-when-not-empty (checked for both employees and child departments)
<code>EmployeeDaoTest</code>	5	<code>EmployeeDAOImpl</code> CRUD: create, <code>findAll</code> , <code>findByDepartmentId</code> , update, deactivate
<code>BudgetDaoTest</code>	5	<code>BudgetDAOImpl</code> CRUD: create, <code>findByDepartmentId</code> , update, delete, <code>findById</code>
<code>DepartmentRecursionTest</code>	3	<code>Department.getHeadcount()</code> recursion: empty department, mixed active/inactive employees, 3-level nested departments
<code>DepartmentTest</code>	6	<code>Department.calculateBudget()</code> , <code>getBudgetVariance()</code> , and the <code>addChild()</code> <code>MAX_DEPTH</code> boundary

The DAO test classes exercise real database round-trips against the schema created by `SchemaInitializer`, so they double as an integration check that the application's SQL is correct, not just that the Java logic is correct in isolation.

6. Building and Running

From source (requires JDK 21+ and Maven):

```
mvn clean install      # compiles, runs all 34 tests
mvn javafx:run         # launches the application
```

- ❖ On first launch, `SchemaInitializer` automatically creates the department, employee, and budget tables if they do not already exist.
- ❖ The application connects to the project's configured MySQL database.
- ❖ A self-contained Windows executable can also be created using `jpackage` with `--type app-image`. It includes a bundled Java runtime, so Java does not need to be installed separately.
- ❖ The packaged build and download instructions are provided in the project README.

7. Team Contributions

Princess Nhyira Addai

- ❖ Connected the Department, Employee, and Budget controllers to their DAO interfaces, enabling the UI to read from and write to the database.
- ❖ Added handling for domain and DAO exceptions, including `IllegalStateException` for the `MAX_DEPTH` limit and `DepartmentNotEmptyException` when deleting non-empty departments.
- ❖ Created a shared `ValidationUtils` class to remove duplicated validation logic and wrote 8 unit tests covering validation and the `MAX_DEPTH` boundary.
- ❖ Fixed database loading issues, including the employee table resetting during navigation and existing employees not being loaded into their department's in-memory list.
- ❖ Styled the application using a dedicated stylesheet.
- ❖ Managed the DAO branch merge into `main` and built/debugged the native `jpackage` executable, resolving a missing-module runtime error using `jdeps`.
- ❖ Authored this report.

J'Isabelle Umuwandimwe

- ❖ Built the JavaFX UI, including FXML views, styling, and the initial controller and model scaffolding.
- ❖ Wrote 26 of the project's 34 unit tests, covering the DAO layer and recursive functionality through `DepartmentDaoTest`, `EmployeeDaoTest`, `BudgetDaoTest`, `DepartmentRecursionTest`, and `DepartmentTest`.
- ❖ Configured Maven packaging using the `maven-shade-plugin` as the basis for the executable build.
- ❖ Finalizing the packaged build and project README.
- ❖ Added a `CurrencyFormatter` utility so budget amounts display in MUR currency format, delete-confirmation and autofill-on-selection in the Budget view, and two additional `DepartmentDAO` tests.